Steps to Run a C/C++ HelloWorld Program in Docker

1. Create a Dockerfile

Create a file named `Dockerfile` in an empty directory with the following content:

`Dockerfile`

```
# Use an official Linux base image
FROM gcc:latest

# Set the working directory inside the container
WORKDIR /app

# Copy your source code into the container
COPY hello.c .

# Compile the C program
RUN gcc -o hello hello.c

# Define the command to run the program
CMD ["./hello"]
```

2. Write the C HelloWorld Program

Create a file named `hello.c` in the same directory as the Dockerfile with the following content:

```
#include <stdio.h>

int main() {
    printf("Hello, World from Docker!\n");
    return 0;
}
```

3. Build the Docker Image

Open a terminal, navigate to the directory containing the `Dockerfile` and `hello.c`, and run:

```
docker build -t hello-world-c .
```

- `-t hello-world-c`: Tags the image with the name `hello-world-c`.
- `.`: Specifies the current directory as the build context.

4. Run the Docker Container

After the image is built, run the container:

```
docker run --rm hello-world-c
```

- `--rm`: Automatically removes the container after it stops.
- The output will display

  ```
  Hello, World from Docker!
  ```

Other useful Docker commands

1. Check if Docker is Running

To check if Docker is running on your system:

```
docker info
```

If Docker is running, it will display system information. If not, you will see an error message.

Alternatively, check the status of the Docker service:

```
systemctl status docker
```

- Look for `Active: active (running)` in the output.

2. Start and Stop Docker Engine

If Docker is not running, you can start or stop the Docker service using the following commands:

- **Start Docker**:

  ```
  sudo systemctl start docker
  ```

- **Stop Docker**:

  ```
  sudo systemctl stop docker
  ```

- **Restart Docker**:

```
sudo systemctl restart docker
```

- **Enable Docker to Start on Boot**:

```
sudo systemctl enable docker
```

- **Disable Docker from Starting on Boot**:

```
sudo systemctl disable docker
```

3. List Running Containers

To list all running containers:

```
docker ps
```

- The output includes the container ID, image, command, status, and other details.

If you want to see all containers (including stopped ones), use:

```
docker ps -a
```

4. List Docker Images

To view all Docker images on your system:

```
docker images
```

This will display the image repository, tag, image ID, creation date, and size.

5. Stop, Start, or Restart a Container

- **Stop a Running Container**:

```
docker stop <container_id_or_name>
```

- **Start a Stopped Container**:

```
docker start <container_id_or_name>
```

- **Restart a Container**:

```
docker restart <container_id_or_name>
```

6. Clean Up Docker

- **Remove a Stopped Container**:

```
docker rm <container_id_or_name>
```

- **Remove an Image**:

```
docker rmi <image_id_or_name>
```

- **Remove All Stopped Containers**:

```
docker container prune
```

- **Remove Unused Images**:

```
docker image prune
```

“permission denied while trying to connect to the Docker daemon”

If you get the following error it’s because the current user does not have the necessary permissions to run Docker commands. Docker requires root privileges or that the user belongs to the docker group. We can fix this issue by adding the current user to docker group.

Step 1: Create the Docker Group (if it doesn’t exist)

```
sudo groupadd docker
```

Step 2: Add Your User to the Docker Group

Replace `<username>` with your username or use `$USER` for the current user:

```
sudo usermod -aG docker $USER
```

Step 3: Log Out and Back In

For the changes to take effect, log out of your current session and log back in. Alternatively, restart your shell session by running:

```
newgrp docker
```

Step 4: Test Docker Without `sudo`

Now, run Docker commands without `sudo`:

```
docker info
```